

What's Next

Phase 1 Handoff & Forward Roadmap

Eclipse Dataspace Components (EDC)

Version	1.0
Date	March 9, 2026
Status	Phase 1 Complete — Entering Phase 2
Supervisors	Mark Klerkx, Toon Teeuwen

Team Members

Benjamin
Juul
Georgi
Dimitar

1. Executive Summary

Phase 1 (Technical) has been completed ahead of schedule. The team has a fully operational dual-node Eclipse Dataspace Components (EDC) environment deployed on Fontys Netlab, with a successful end-to-end mock data transfer validated. The strategic decision has been made to proceed exclusively with EDC, dropping FIWARE.

This document captures what was accomplished, what the current state of the infrastructure looks like, and the concrete next steps for Phase 2 (Functional) and Phase 3 (Finalization).

2. Phase 1 Accomplishments

2.1 Infrastructure Deployed

The team deployed two isolated EDC instances (Data Provider and Data Consumer) onto a single Fontys Netlab Ubuntu VM (192.168.120.12), using Kubernetes (K3s) namespaces and separate PostgreSQL 14.2 databases for strict logical separation.

Node	UI Port	Mgmt API	DSP Port
Provider	3000	11002	11003
Consumer	4000	12002	12003

2.2 Technical Challenges Resolved

- Bypassed Kubernetes LoadBalancer/Ingress conflicts by binding pod ports directly to the VM's public interface using `kubectl port-forward --address 0.0.0.0`.
- Diagnosed and resolved UI FetchError / CORS issues by configuring EDC pods with the exact VM IP and accessing dashboards directly via Fontys Cisco AnyConnect VPN.
- Eliminated the need for restrictive SSH tunnels for daily development.

2.3 End-to-End Validation

A complete mock data transfer was successfully executed, validating the full pipeline:

- **Asset creation** via a dummy HTTP REST API (jsonplaceholder).
- **Policy definition** using an “allow-all” contract.
- **Contract negotiation** initiated from the Consumer node.
- **Data pull** with the payload successfully received by the Consumer.

2.4 Framework Decision: EDC Only

In line with the Phase 1 risk mitigation strategy outlined in the project plan, the team evaluated the overhead of maintaining both EDC and FIWARE. The decision was made to drop FIWARE and proceed exclusively with Eclipse Dataspace Components.

Rationale:

- EDC is already fully deployed and validated on Netlab.

- Maintaining two frameworks would split limited development resources without proportional benefit.
- Dropping FIWARE frees up Benjamin to contribute to backend engineering and Phase 2/3 software tasks.

3. Current State Overview

Component	Status	Notes
EDC Provider Node	Complete	Running on K3s default namespace
EDC Consumer Node	Complete	Running on K3s consumer namespace
VPN / Network Access	Complete	Cisco AnyConnect, no SSH tunnels needed
Mock Data Transfer	Complete	End-to-end validated with allow-all policy
Identity Module (IAM)	Blocked	Currently using soivity-mock-iam placeholder
ODRL Policy Enforcement	Not Started	Allow-all only; real policies needed
Framework Comparison Report	Not Started	Deliverable 3 — needs to be drafted
Real-World Backend Services	Not Started	Phase 3 scope

4. What's Next: Roadmap

4.1 Immediate: Phase 1 Wrap-Up

Priority	Task	Owner	Target
HIGH	Draft framework comparison and evaluation report (Deliverable 3). Document the technical justification for selecting EDC and dropping FIWARE.	Benjamin	Week of Mar 9

4.2 Phase 2: IAM & Security Implementation

The current setup relies on soivity-mock-iam, a placeholder identity module. Phase 2 replaces this with a production-grade identity framework built on W3C standards. This is the highest-priority work stream.

#	Task	Owner	Priority	Target
---	------	-------	----------	--------

1	Generate and host did:web documents for both Provider and Consumer connectors.	Juul	HIGH	Mar 23 – Apr 3
2	Deploy an Identity Provider (IdP) / Trust Anchor service capable of issuing signed Verifiable Credentials to participating nodes.	Juul	HIGH	Apr 6 – Apr 17
3	Write ODRL JSON-LD policy scripts enforcing attribute-based data governance (e.g., verifying specific VC claims before allowing data transfer).	Juul + Dimitar	HIGH	Apr 20 – May 1
4	Replace sovity-mock-iam with the production IAM configuration and validate DID-authenticated data exchange end-to-end.	All	HIGH	May 4 – May 15
5	Define and execute use case demonstration with full VC/ODRL governance.	Georgi + Dimitar	HIGH	May 18 – May 22

Phase 2 Exit Criteria:

- sovity-mock-iam is fully replaced with real DID + VC-based identity.
- At least one use case is demonstrated with policy-governed data transfer.
- Combined project plan for IAM and Marketplace is delivered.

4.3 Phase 3: Application & Extension Development

With the infrastructure and security layers in place, Phase 3 focuses on replacing dummy services with real-world applications and extending EDC where needed.

#	Task	Owner	Priority	Target
1	Set up a local Java/Gradle development environment and write custom EDC extensions (e.g., MQTT connectors for IoT, custom webhook triggers).	Georgi	MEDIUM	May 25 – Jun 5
2	Build Provider-side backend API serving realistic data (e.g., smart city metrics, positive energy district data, traffic conditions) using Python FastAPI or Node.js.	Benjamin	MEDIUM	May 25 – Jun 12
3	Build Consumer-side application/dashboard that programmatically ingests EDC-transferred data and visualizes it for end users.	Dimitar	MEDIUM	Jun 1 – Jun 19

4	Harden security configurations, finalize documentation, and prepare project hand-off package.	All	HIGH	Jun 22 – Jun 26
---	---	-----	-------------	-----------------

5. Updated Role Assignments

With FIWARE dropped, team responsibilities have been redistributed to focus on EDC-exclusive development:

Member	Phase 2 Focus	Phase 3 Focus
Benjamin	Framework comparison report; support IAM integration	Provider-side backend API development
Juul	IAM Lead: DID, Verifiable Credentials, Trust Anchor deployment	IAM hardening & documentation
Georgi	Use case design & validation; ODRL support	Custom Java/Gradle EDC extensions
Dimitar	ODRL policy scripting; combined IAM + Marketplace plan	Consumer-side dashboard development

6. Open Questions & Decisions Needed

#	Question	Decision By	Needed By
1	Which real-world use case(s) will be implemented? (e.g., smart city data, IoT sensor streams, energy metrics)	Team + Supervisors	Mar 23
2	Which IdP/Trust Anchor implementation to use? (e.g., Keycloak-based, custom, or FIWARE-derived components)	Juul + Supervisors	Mar 23
3	Tech stack for backend services: Python FastAPI or Node.js?	Benjamin + Dimitar	May 22
4	Does the scope of custom EDC Java extensions justify the setup cost, or can we stay within HTTP-based data sources?	Georgi + Supervisors	May 22

7. Deliverables Tracker

#	Deliverable	Phase	Status	Owner
1	Functional EDC dual-node environment	Phase 1	Complete	All
2	Framework comparison & evaluation report	Phase 1	Not Started	Benjamin
3	Updated technical documentation	Phase 1–3	In Progress	All

4	Production IAM (DID + VC + Trust Anchor)	Phase 2	Not Started	Juul
5	ODRL policy-governed data exchange demo	Phase 2	Not Started	Juul + Dimitar
6	Combined IAM + Marketplace project plan	Phase 2	Not Started	Dimitar
7	Custom EDC Java extensions	Phase 3	Not Started	Georgi
8	Real-world backend services (Provider + Consumer)	Phase 3	Not Started	Benjamin + Dimitar
9	Final hand-off package & retrospective	Phase 3	Not Started	All